ФСП Карьера — документация

Платформа подбора ИТ-специалистов для Федерации спортивного программирования: кандидат получает **категорию** (специализация × грейд) по результатам теста, а работодатель **сам находит** подходящего человека в категоризированной базе и отправляет ему предложение с вилкой зарплаты.

Содержание

1. Соответствие ТЗ
2. Архитектура
3. Роли, безопасность, персональные данные
4. Категоризация и тестирование
5. Подбор (матчинг)
6. Валидация
7. Интеграция с ФСП
8. API
9. Библиотеки и версии
10. Запуск
11. Монетизация, награды, тренировка
12. Развитие за пределами MVP
13. Известные ограничения
14. Изменения сборки 2026.10.11-2

1. Соответствие ТЗ

Требование ТЗ	Как реализовано	Где
Кабинеты кандидата и работодателя, разделение ролей	Роли «кандидат» и «работодатель» (плюс служебная роль «модератор» для проверки прокторинга), проверка роли на каждом методе API (403 при чужой роли), отдельные разделы интерфейса	`app/apidoc.py` (параметр `auth=`), `app/routers/*`
Регистрация по e-mail с подтверждением	Письмо со ссылкой, вход до подтверждения закрыт; в `DEV_MODE` ссылка возвращается в ответе	`routers/auth.py`
Согласие на обработку ПДн (152-ФЗ)	Регистрация невозможна без `consent_pd`; экспорт и удаление данных; отдельное согласие на публикацию профиля работодателям	`schemas.RegisterIn`, `candidate/export`, `auth/account`
Обязательный путь: опрос → выбранный грейд → тест	Тест нельзя начать без опроса (409 `survey_required`); категория присваивается **только** результатом теста	`domain.start_attempt`, `domain.apply_outcome`
Категории «специализация × грейд», не самоописанное резюме	5 специализаций × 4 грейда = 20 категорий; технологии выбираются из каталога на 159 позиций; резюме в ранжировании не участвует	`taxonomy.py`, `matching.py`
Ранжирование внутри категории по подтверждённому профилю	`P = 0.35·тест + 0.30·подтверждённый стек + 0.25·ФСП + 0.10·актуальность`, у каждого кандидата — объяснение «почему»	`matching.py`, §5
Тест устойчив к передаче заданий, сопоставимая сложность, объективность	Параметрическая генерация заданий под каждую попытку, ответы считаются кодом, фиксированный чертёж сложности (20 заданий, шкала 1–5), проверка на сервере, **прокторинг** (камера, полноэкранный режим, журнал событий, модерация)	`testgen.py`, `testgen_hard.py`, `engine.py`, `proctor.py`, §4
Нет принудительного понижения грейда; смена не чаще 1 раза в 3 месяца	Провал не меняет категорию, только предлагает ступень ниже; смена грейда/специализации — раз в 90 дней (настраивается), первое присвоение и 24-часовое «окно калибровки» свободны	`domain.cooldown_state`

Требование ТЗ	Как реализовано	Где
Необязательные регулярные микрозадачи работодателя	Работодатель публикует задание (до 2 в неделю), кандидат отвечает, работодатель оценивает; оценка влияет на актуальность профиля	`routers/employer.py`, `routers/candidate.py`
Главное: работодатель инициирует контакт (35%)	Банк кандидатов с фильтрами → карточка → приглашение с **обязательной вилкой «от–до» в рублях**	`employer/invites`, `schemas.InviteIn`
Контакты раскрываются только после согласия кандидата	Работодатель видит псевдоним `C-XXXXXX`; имя и контакты — только при принятом приглашении или собственном отклике кандидата	`domain.revealed_ids`, `employer_card`
Статусы отправлено / просмотрено / принято / отклонено	Жизненный цикл приглашения, «просмотрено» ставится при открытии кандидатом; отзыв приглашения работодателем	`routers/candidate.py`
Привязка к ФСП ID, кандидаты без истории ФСП	Привязка по совпадению e-mail с реестром; без ФСП кандидат полноценно участвует (нулевая составляющая, не штраф к категории)	`services/fsp.py`, §7
Разбор PDF-резюме и генерация PDF	Эвристический парсер (e-mail, телефон, ФИО, стек, опыт, роли) предзаполняет профиль; PDF профиля с блоком «Подтверждено платформой»	`services/resume.py`
OpenAPI / Swagger	`/openapi.json` (OpenAPI 3.0.3, генерируется из тех же схем, что валидируют запросы) и интерфейс `/docs`	`app/apidoc.py`
Docker, открытый код	`Dockerfile`, `docker-compose.yml`; код без закрытых зависимостей	корень репозитория
Обязательная документация	Этот документ: §2 архитектура, §4 механика тестов и устойчивость, §5 матчинг, §6 валидация, §7 интеграция, §8 API, §9 библиотеки, §10 запуск	`docs/`

---

2. Архитектура

2.1 Функциональная схема



2.2 Компоненты

Компонент	Ответственность
`app/apidoc.py`	Микрослой над Flask: декоратор `api.route(...)` одновременно задаёт маршрут, роль, схему тела/параметров (Pydantic) и описание для OpenAPI. Документация не может разойтись с кодом — она строится из него (тест `test_openapi_covers_all_routes`).
`app/schemas.py`	Все входные модели и их правила (вилка зарплаты 1 000–10 000 000 ₽ и `до ≥ от`, пароль: буквы+цифры, обязательность согласия и т.д.).

Компонент	Ответственность
`app/security.py`	Пароли — scrypt с индивидуальной солью, сравнение за постоянное время; JWT (HS256) с TTL.
`app/db.py`	SQLite, схема, тонкие помощники `q1/qa/ex`. Таблицы: `users, candidates, employers, test_attempts, proctor_events, proctor_snapshots, moderation_log, grade_history, needs, vacancies, invites, applications, microtasks, microtask_assignments`.
`services/testgen.py`, `services/testgen_hard.py`	Около 80 параметрических семейств заданий уровней 1–5 (в тест одной специализации попадает 34–36 возможных семейств) и пул концептуальных вопросов (§4).
`services/proctor.py`	Прокторинг: журнал событий, взвешивание риска, автоматическое прекращение, очередь и решения модератора, срок хранения снимков (§4.7).
`techcatalog.py`	Каталог технологий (159 позиций, 12 групп) с синонимами для разбора PDF-резюме.
`services/engine.py`	Сборка теста по «чертежу» сложности, проверка ответов, подтверждённый стек, флаги подозрительности.
`services/matching.py`	Коэффициент категории, сила профиля, объяснение.
`services/domain.py`	Правила, где сходятся все данные: кто что видит, когда можно начать тест, как присваивается категория.
`services/fsp.py`	Контракт реестра ФСП и расчёт силы достижений.
`services/resume.py`	Разбор PDF-резюме и генерация PDF профиля (reportlab, шрифт DejaVu — кириллица).
`app/static/*`	SPA на чистом JS: хеш-роутинг, делегирование событий, без CDN и сборки — работает офлайн. Единая тёмно-фиолетовая тема с градиентом (`app.css`), `proctor.js` — клиентская часть прокторинга, `moderator.js` — кабинет модератора. Подвал с описанием ФСП строится из `data/fsp_about.json`.
`scripts/seed.py`, `simulate.py`, `validate.py`	Демо-данные, модель синтетического кандидата, процедура валидации.

2.3 Почему такой стек

Хакатонное окружение, в котором велась разработка, закрывало PyPI, поэтому вместо FastAPI/SQLAlchemy использованы Flask, Pydantic и SQLite из стандартной библиотеки; недостающее (OpenAPI, валидация ответов-ошибок) реализовано в `apidoc.py` на ~200 строках. Для продакшена SQLite заменяется на PostgreSQL изменением слоя `db.py` — SQL в проекте стандартный, без диалектных расширений.

---

3. Роли, безопасность, персональные данные

- **Роль «модератор».** Видит очередь отмеченных попыток, журнал и снимки с камеры, принимает решение (§4.7). Создаётся только командой `python scripts/create_moderator.py` — самостоятельно зарегистрироваться модератором нельзя.
- **Разделение ролей.** Роль проверяется на сервере для каждого маршрута; кандидат не может вызвать методы работодателя и наоборот. Данные другого пользователя недоступны по идентификатору (проверка владельца: приглашение, вакансия, попытка теста — отдельные тесты изоляции).
- **Псевдонимность.** Работодатель видит кандидата как `C-XXXXXX`. Фамилия, контакты и точный город скрыты до принятия приглашения; город показывается, только если кандидат разрешил, и по нему же работает фильтр (нельзя вычислить скрытое значение фильтрацией).
- **Публикация.** Кандидат попадает в банк, только если у него есть категория **и** согласие на публикацию. Настройки приватности: показывать стек / достижения ФСП / опыт / город; можно скрыться из банка, оставив возможность откликаться на вакансии.
- **Контакты.** Раскрываются в двух случаях: кандидат принял приглашение; кандидат сам откликнулся на вакансию. Отклонённое приглашение контакты не раскрывает.
- **Аутентификация.** Подтверждение e-mail, блокировка входа после 5 неудач за 15 минут, JWT с ограниченным сроком, заголовки безопасности (`X-Content-Type-Options` , `X-Frame-Options: DENY` , `Referrer-Policy`); весь вывод в интерфейсе экранируется.
- **152-ФЗ.** Согласие фиксируется при регистрации (дата хранится), отдельный переключатель публикации, выгрузка всех данных (`GET /api/candidate/export`), удаление аккаунта с каскадом (`DELETE /api/auth/account` , подтверждается паролем).
- **Антиспам.** Работодатель — не более 25 приглашений в сутки (`INVITES_PER_DAY`), микрозадач — 2 в неделю.

---

4. Категоризация и тестирование

4.1 Путь кандидата

1. **Опрос:** отрасль, специализация, технологии, самооценка грейда, формат работы. Самооценка нужна только как *запрос* — какой тест предложить.
2. **Выбор грейда:** стажёр / junior / middle / senior.
3. **Тест под прокторингом** на выбранную специализацию и грейд (20 заданий, 90 минут, камера и полноэкранный режим, согласие на обработку изображения — §4.7).
4. **Результат → категория.** Зачёт присваивает категорию и подтверждает стек. Незачёт **ничего не отнимает**: платформа предлагает пройти тест на ступень ниже (а при результате ≥ 85% — на ступень выше).

4.2 Генерация заданий

Банк фиксированных вопросов быстро «утекает» (его выкладывают в чаты и пересказывают). Поэтому основная часть заданий **не хранится, а порождается**: семейство задаёт шаблон условия,

а **числа, строки, структуры данных и варианты ответа выбираются заново для каждой попытки** (зерно — `seed` попытки). Правильный ответ вычисляется кодом — выполнением сгенерированной программы, SQL-запроса к сгенерированной таблице, `ipaddress` для CIDR и т.п.; поэтому он не может «разойтись» с условием.

Семейства — около 80 параметрических (24 общих и 56 специализированных), шесть групп по сложности:

- **Общие, уровни 1–3:** трассировка Python-кода, O-нотация, бинарный поиск, подсчёт обменов сортировки, структуры данных, SQL по сгенерированной таблице, регулярные выражения, битовые операции, графы, динамическое программирование, CIDR.
- **Общие, уровни 4–5 (`testgen_hard.py`):** трассировка с рекурсией и замыканиями, рекуррентные соотношения, остовное дерево / пути / максимальный поток, ДП (LCS, расстояние редактирования, подмножества, LIS), SQL с оконными функциями и `HAVING` (результат считает настоящий SQLite), **все возможные чередования потоков** при гонке данных, битовые трюки, CIDR-агрегация, регулярные выражения со жадностью, хеш-таблицы с коллизиями, вероятность, планировщики (Round Robin, FIFO/LRU/OPT), модульная арифметика.
- **Backend:** кворумы репликации, уровни изоляции транзакций, консистентное хеширование, скользящее окно, очередь M/M/1, фильтр Блума.
- **Frontend:** event loop, flexbox, `em` / `rem`, обновления состояния React, `parseInt` /приведение типов, контраст WCAG.
- **Data:** AUC-ROC, k-means, линейная регрессия, информационный выигрыш, градиентный спуск, наивный Байес, корреляция.
- **DevOps:** упаковка по ресурсам, бюджет ошибок SLO, подсети VPC, слои Docker и кэш сборки, порядок правил файрвола, smooth weighted round-robin.
- **QA:** попарное тестирование, граничные значения, покрытие условий, мутационное тестирование, нестабильные тесты.

Ответ на каждое семейство — **число**, посчитанное кодом из сгенерированных данных (часто сумма по нескольким подзадачам, чтобы ответ нельзя было угадать: доля самого частого ответа по разным сидам в проверке ≤ ~0,2). К семействам добавляется небольшой **пул концептуальных вопросов** (не более 3 из 20, пониженный вес выбора) — единственная статичная часть.

4.3 Сопоставимая сложность и объективность

- **Чертёж сложности.** Шкала уровней — 1–5, тест — 20 заданий. «Ядровой» уровень грейда: стажёр — 2, junior — 3, middle — 4, senior — 5.

Грейд	Состав теста (20 заданий)
стажёр	3×ур.1 + 8×ур.2 + 7×ур.3 + 2×ур.4
junior	3×ур.2 + 8×ур.3 + 7×ур.4 + 2×ур.5
middle	3×ур.3 + 9×ур.4 + 8×ур.5
senior	2×ур.3 + 4×ур.4 + 14×ур.5

  Два кандидата одного грейда получают разные задания **одинаковой структуры сложности**. Вес задания равен его уровню; не более двух заданий одного семейства в тесте.

- **Критерий зачёта:** взвешенная доля баллов не ниже порога грейда — **65%** (стажёр, junior), **70%** (middle), **80%** (senior) — **и** верно решена половина «ключевых» заданий ядрового уровня (для senior — две трети). Пороги подобраны на модели (§6) и подлежат калибровке на реальных данных; общий порог можно задать переменной `PASS_THRESHOLD`.
- **Объективность:** проверка полностью автоматическая и детерминированная; числовые ответы принимаются с допуском, заданным в семействе; разбор с верными ответами показывается *после* сдачи — он ничего не раскрывает другим, потому что у другого человека будут другие числа.

4.4 Почему передача заданий не работает

Сценарий	Что происходит	Мера
«Скинь мне вопросы и ответы»	У другого кандидата другие числа/данные: чужие ответы неверны. Модель: подстановка *идеальных* ответов друга даёт в среднем 3–4% (§6, раздел E).	Генерация по попытке
Утечка условий в чат	Условие без чужих чисел бесполезно; «слитый» банк из 300 попыток закрывает в среднем 8–24% заданий нового теста, ни разу не весь тест (§6, раздел A). Остаточный риск — пул концептуальных вопросов (около 15 на специализацию) и семейства уровня 1 с малым пространством ответов.	Ограничение пула ≤ 3 вопросов, расширенные пространства параметров
Повторные попытки «на разведку»	Не более 5 попыток в сутки, одна активная, таймер на сервере (дедлайн хранится в БД, а не в браузере); каждая попытка — новый вариант	Лимиты
Подбор «наугад»	Числовые ответы не угадываются; вероятность зачёта при случайных ответах ≈ 0% (§6, E)	Числовой ввод, веса
Подсказка в реальном времени (человек решает за кандидата)	**Технически не устраняется любым онлайн-тестом.** Снижаем стимул и повышаем заметность: прокторинг (§4.7), быстрые ответы на сложные задания, флаги `too_fast` и `late`; при отметке вес профиля в подборе снижается на 15% до решения модератора, работодатель видит пометку	Прокторинг, модерация, вес профиля

4.5 Ограничения смены грейда

- Повторная попытка **той же** категории разрешена всегда (в пределах суточного лимита).

- Первое присвоение категории свободно; затем 24 часа «окна калибровки», когда можно один раз пересмотреть выбор (человек мог ошибиться при самооценке).
- Дальше смена грейда или специализации — не чаще раза в `GRADE_COOLDOWN_DAYS` (по умолчанию 90 ≈ 3 месяца). Интерфейс показывает дату, когда смена станет доступна.
- Провал теста никогда не понижает грейд.

4.6 Подтверждённый стек

Стек определяется не списком, который назвал кандидат, а **результатом по темам**: технология считается подтверждённой, если по связанной теме верно ≥ 66% (при ≥ 2 заданиях) либо решено задание сложности ≥ 4. Подтверждаемые технологии берутся из каталога (159 позиций); связь «тема → технологии» — `taxonomy.TOPIC_STACK`. Работодатель видит отдельно «подтверждено тестом» и «заявлено кандидатом (не подтверждено)».

4.7 Прокторинг

Тест проходит под наблюдением. Это сочетание **браузерных событий**, **снимков веб-камеры** и **решения человека-модератора**; автоматика только отмечает и в крайнем случае прекращает попытку, окончательное решение об аннулировании принимает модератор.

Допуск. Тест нельзя начать без согласия на прокторинг (`consent_proctoring`, 422 `proctoring_consent_required`) и без включённой камеры (`camera_ready`, 422 `camera_required`; отключается переменной `PROCTOR_CAMERA_REQUIRED=0`). Перед заданиями кандидат проходит «шлюз»: камера + полноэкранный режим; выйдя из полноэкранного режима, он снова видит шлюз.

Что фиксирует браузер (`static/proctor.js`, пакеты на `POST /api/candidate/tests/{id}/proctor` каждые 8 секунд):

Событие	Вес	Потолок вида
Уход на другую вкладку / потеря фокуса окна	3 / 1	24 / 6
Выход из полноэкранного режима	4	24
Копирование / вырезание / вставка	2 / 2 / 4	8 / 8 / 12
Инструменты разработчика (F12, Ctrl+Shift+I, подозрительная разница размеров окна)	5	15
Печать, запрещённые сочетания, контекстное меню	3 / 1 / 1	6 / 4 / 3
Камера закрыта или кадр тёмный	5	15
Нет лица / несколько лиц в кадре (если браузер поддерживает `FaceDetector`)	3 / 6	12 / 18

Раз в 30 секунд делается снимок (JPEG 320×240, сжатие 0,55), он уходит на сервер.

Что решает сервер (`services/proctor.py`; клиенту доверять нельзя, поэтому часть видов формирует только сервер):

- Время события ставит сервер; допускаются только известные виды, мета-данные усекаются.
- **Вторая вкладка / другое устройство** с той же попыткой: другой идентификатор вкладки → событие `multi_session` (вес 8).
- **Слишком быстрые ответы** на задания сложности ≥ 4 (< 6 с, не менее трёх) → `fast_answers`.
- **Дыры в съёмке:** если за время теста снимков заметно меньше ожидаемого → `camera_gaps`; камера не передаётся → `camera_off`.
- Суммарный **риск** — сумма весов с потолком на каждый вид (шум одного вида не «топит» кандидата).
- Риск ≥ **12** → попытка отмечена и попадает в очередь модератора; вес профиля в подборе ×0,85 до решения.
- Риск ≥ **45** → попытка **немедленно прекращается**: категория не меняется, кандидат видит причину.

Модерация (роль `moderator`, `routers/moderator.py`, интерфейс `#/m`): очередь с сортировкой по риску, журнал событий, галерея снимков, комментарий и решение:

- *Снять отметку* — результат остаётся, вес профиля восстанавливается.
- *Аннулировать* — попытка аннулируется; если категория была присвоена именно этой попыткой (`candidates.category_attempt_id`), она снимается, и кандидату нужно пройти тест заново. Решение пишется в `moderation_log`.

Персональные данные (152-ФЗ). Снимки — изображение человека, поэтому: согласие запрашивается отдельно перед каждым тестом; снимки видит только модератор; хранятся `PROCTOR_RETENTION_DAYS` (30 по умолчанию) и затем удаляются при запуске приложения (кроме попыток, ждущих решения); кандидату в разборе показывается, что именно записано.

Честно об ограничениях. Всё, что делает браузер, можно обойти на уровне ОС: вторая камера, второй компьютер, подсказчик за спиной, виртуальная камера. Прокторинг не доказывает честность — он делает нечестность дорогой и заметной, а сомнительные случаи отдаёт человеку. Для Safari/iOS полноэкранный режим недоступен (шлюз всё равно пускает, но событие не фиксируется); `FaceDetector` пока есть не во всех браузерах — без него проверка лиц не выполняется, остальное работает. Видеопоток на сервер не передаётся — только отдельные снимки.

---

5. Подбор (матчинг)

Работодатель описывает потребность: специализация, грейд, стек. Далее:

1. **Рекомендованные категории.** Показываются категории из банка с коэффициентом K ≥ 0.2 и числом кандидатов в каждой.

2. **Оценка кандидата:**

```
итог = K(категория) × (0.15 + 0.85 × P)
P    = 0.35·тест + 0.30·подтверждённый_стек + 0.25·ФСП + 0.10·актуальность
```

- **K(категория)** — главный множитель: совпадение 1.0; соседний грейд той же специализации 0.55–0.60; через ступень 0.20–0.25; смежная специализация — вес связи (например backend↔devops 0.35) × коэффициент грейда. Кандидат «не той» категории не обгонит подходящего за счёт сильного профиля.
- **тест** — нормированный результат: (доля − 0.65) / 0.35.
- **подтверждённый_стек** — доля запрошенных технологий, подтверждённых тестом.
- **ФСП** — «сила» участия: 35% объём, 40% лучшее относительное место, 25% давность (окно 3 года). Нет истории → 0, но **категория не страдает**: множитель K от ФСП не зависит.
- **актуальность** — давность активности (окно 120 дней) и оценки микрозадач.
- Флаг подозрительного прохождения умножает P на 0.85.

3. **Объяснение.** Для каждого кандидата возвращается список факторов с числом и текстом («подтверждено 2 из 3: python, sql», «топ-9.4% на Открытом кубке ФСП», «истории ФСП нет — не снижает категорию»). В интерфейсе — кнопка «почему».

4. **Фильтры** (специализация, грейд, стек, минимальный тест, город, «есть ФСП») сужают выдачу; счётчик «Найдено» отдельно хранит размер исходной выборки, чтобы работодатель видел, сколько кандидатов отсекли фильтры.

Самоописанное резюме (опыт, роли, soft skills) показывается в карточке с пометкой «заполнено самим кандидатом и не влияет на ранжирование».

---

6. Валидация

6.1 Что и зачем проверяем

У нас нет реальных кандидатов и разметки грейдов, поэтому проверяется **конвейер** (генерация → проверка → порог → категория → подбор) на синтетической популяции. Для каждого вопроса ТЗ — своя проверка:

Вопрос	Проверка (раздел результатов)
Тесты нельзя переслать?	Доля совпадающих заданий между двумя попытками; покрытие нового теста «слитым» банком из N попыток (A)
Варианты одного грейда одинаково сложны?	Ожидаемая доля баллов для кандидата фиксированной способности по 400 вариантам: среднее и разброс (B)
Тест различает грейды?	Матрица «истинный грейд × проходимый тест» (C)
Результат воспроизводим?	Повторное прохождение: согласие зачёт/незачёт, к Коэна, корреляция долей баллов; устойчивость присвоенной категории (D)
Тест устойчив к недобросовестным способам?	Случайные ответы; списывание идеальных ответов друга (E)
От допущений модели зависят выводы?	Анализ чувствительности (G)

Вопрос	Проверка (раздел результатов)
Подбор лучше, чем по резюме?	P@10 и nDCG@10 против ранжирования по самоописанию при разной доле завышений (F)

6.2 Модель кандидата

`scripts/simulate.py` : у кандидата скрытая способность θ; вероятность верного ответа на задание уровня L — логистическая функция от разности θ и сложности уровня (+ угадывание 25% на заданиях с выбором). Специалист грейда *g* имеет θ = g + 2.6 (+ шум 0.35); на шкале 1–5 это значит, что он **уверенно решает задания своего ядрового уровня g+2** и решает примерно две трети заданий уровнем выше — это допущение-определение «что такое грейд». Анализ чувствительности (раздел G) показывает, как меняются выводы при сдвиге 2.0–2.9.

Запуск: `python3 scripts/validate.py` (≈1 минута, детерминирован, seed = 2026) → `docs/VALIDATION_RESULTS.md` , `docs/validation_results.json` .

6.3 Результаты

Сгенерировано `scripts/validate.py` , seed=2026. **Это модель, а не реальные кандидаты** — см. оговорки в §6.5 документации.

A. Уникальность и утечка

Категория	Заданий	Совпадающих заданий между двумя попытками	Совпадение «условие+ответ»	Видимое «слитым» банком из 300 попыток
backend/middle	20	0.16%	0.11%	9.7% (макс. 20%)
frontend/junior	20	0.16%	1.34%	13.8% (макс. 35%)
data/senior	20	0.31%	0.21%	11.9% (макс. 30%)
devops/trainee	20	0.29%	0.14%	15.7% (макс. 35%)
qa/middle	20	0.21%	0.15%	14.5% (макс. 35%)

B. Эквивалентность вариантов одного грейда

Грейд	Ожидаемая доля баллов (ср.)	SD между вариантами	мин	макс
trainee	0.751	0.0066	0.746	0.787
junior	0.766	0.0043	0.764	0.79
middle	0.843	0.0022	0.842	0.853
senior	0.94	0.0007	0.939	0.942

C. Дискриминация: доля зачёта (строка — истинный грейд, столбец — проходимый тест)

	trainee	junior	middle	senior
trainee	73%	9%	0%	0%
junior	100%	84%	19%	1%
middle	100%	100%	90%	47%
senior	100%	100%	100%	96%

D. Надёжность (повторное прохождение, тот же уровень, новые варианты)

Грейд	Зачёт	Согласие	к Коэна	r (доля баллов)
trainee	72%	75%	0.36	0.465
junior	83%	80%	0.297	0.416
middle	89%	86%	0.306	0.397
senior	96%	93%	0.152	0.179

Устойчивость присвоенной категории при повторной «лесенке» тестов: **68%**.

E. Атаки

Грейд	Случайное угадывание: зачёт	Списывание идеальных ответов друга: зачёт	…средняя доля баллов
trainee	0.00%	0.00%	3%
junior	0.00%	0.00%	2%
middle	0.00%	0.00%	1%
senior	0.00%	0.00%	1%

G. Чувствительность к допущению о способности (сдвиг θ; базовое значение — 2.6)

Сдвиг	Грейд	Зачёт своего теста	Зачёт теста на ступень выше	Зачёт теста на ступень ниже
2.0	trainee	31%	1%	—
2.0	junior	36%	3%	89%
2.0	middle	48%	9%	96%
2.0	senior	73%	—	97%
2.3	trainee	52%	4%	—
2.3	junior	59%	7%	98%
2.3	middle	73%	23%	98%
2.3	senior	91%	—	100%
2.6	trainee	78%	12%	—
2.6	junior	78%	19%	99%

Сдвиг	Грейд	Зачёт своего теста	Зачёт теста на ступень выше	Зачёт теста на ступень ниже
2.6	middle	91%	41%	100%
2.6	senior	95%	—	100%
2.9	trainee	93%	33%	—
2.9	junior	95%	40%	100%
2.9	middle	95%	64%	100%
2.9	senior	99%	—	100%

F. Подбор: платформа против ранжирования по самоописанному резюме

Условие — доля кандидатов, завысивших грейд в самооценке. Ground truth — истинные специализация и грейд.

Завысили грейд	Метод	Precision@10 (точная категория)	nDCG@10
0%	Платформа (категория по тесту + подтверждённые данные)	0.645	0.76
	Базовая линия (самоописание)	0.906	0.936
25%	Платформа (категория по тесту + подтверждённые данные)	0.663	0.755
	Базовая линия (самоописание)	0.687	0.791
50%	Платформа (категория по тесту + подтверждённые данные)	0.65	0.769
	Базовая линия (самоописание)	0.53	0.693
75%	Платформа (категория по тесту + подтверждённые данные)	0.635	0.74
	Базовая линия (самоописание)	0.307	0.537

Популяция: ≈657 кандидатов с категорией, 150 потребностей; при 25% завышений категория по тесту совпала с истинной у 75% кандидатов (остальные — шум теста и «везение» на ступень выше).

6.4 Как читать результаты (выводы и найденные слабости)

- **Передача заданий не помогает.** Два случайных теста совпадают по заданиям на доли процента; списывание идеальных ответов друга даёт 1–3% баллов; случайные ответы не проходят порог.

- **Утечка не нулевая.** «Слитый» банк из 300 попыток закрывает в среднем 10–16% заданий нового теста, максимум — 20–35%; полностью закрытых тестов нет. Самые уязвимые — пул концептуальных вопросов и нижние уровни; пул нужно расширять и мониторить.
- **Валидация находила дефекты, и мы их исправляли.** Первая версия: у senior не было уровня выше 4, и его тест проходили ≈ 70% специалистов middle. Ввели уровень 5, усиленное условие по ключевым заданиям и пороги по грейдам; после этого middle проходит тест senior в **≈ 47%** случаев (раздел C) — **это всё ещё много**: на модели тест senior различает хуже остальных, потому что выше уровня 5 заданий нет. Требуется расширение банка и калибровка на реальных данных.
- **Надёжность умеренная.** к повторного прохождения 0.15–0.36, корреляция долей баллов 0.18–0.47: результат на границе порога колеблется, особенно у senior (потолок шкалы сжимает разброс). Практический вывод: повторная попытка той же категории разрешена, а решение о смене категории не должно основываться на одном результате вблизи порога.
- **Подбор: честная картина.** Если *все* кандидаты честно оценивают грейд, ранжирование по резюме точнее (оно получает правильный ответ «бесплатно»). Когда доля завышающих самооценку достигает 50% и выше, платформа начинает выигрывать, и разрыв растёт. Это и есть заявленная ценность: категория не зависит от того, что человек о себе написал.
- **Что валидация не охватывает:** сам прокторинг. Модель не содержит «нечестных» кандидатов с подсказкой, поэтому пороги риска (12 / 45) и веса событий — экспертные значения, а не результат калибровки; их нужно пересмотреть по данным пилота (доля ложных отметок, решения модераторов).

6.5 Ограничения валидации (что делать с реальными данными)

1. Все цифры — **на модели**; они проверяют логику, а не то, что задания измеряют именно «грейд». Содержательную валидность можно показать только на реальных кандидатах с экспертной разметкой.
2. Допущение $\theta = g + 2.6$ произвольно (раздел G показывает зависимость). По реальным данным сдвиг, пороги зачёта (0.65 / 0.70 / 0.80) и доли в чертеже подбираются заново.
3. План калибровки: (а) пилот на 100–200 участниках ФСП с известными результатами соревнований; (б) анализ каждого семейства по доле верных ответов и корреляции с общим результатом, исключение «слишком лёгких/слишком шумных» семейств; (в) экспертная проверка 5–10% заданий на однозначность условия; (г) пересчёт порога по кривой «доля зачёта при известных грейдах»; (д) регулярный мониторинг утечки — частота совпадений ответов в попытках разных людей.
4. Модель не учитывает осознанные «ферм-аккаунты» и совместное решение в реальном времени — прокторинг (§4.7) делает это заметнее, но не исключает.

---

7. Интеграция с ФСП

7.1 Контракт

Документации API ФСП в рамках хакатона нет, поэтому определён минимальный контракт реестра, на котором работает платформа. Запись участника:

```
        {
          "fsp_id": "FSP-100001",
          "email": "user@example.org",
          "full_name": "Имя Фамилия",
          "events": [
            {"id": 10, "title": "Открытый кубок ФСП", "type": "contest",
             "date": "2026-06-04", "place": 17, "participants": 200, "points": 46}
          ]
        }
```

Интерфейс адаптера один: `registry.get(fsp_id) → запись | None` (`services/fsp.py`). Реализация MVP — `FileRegistry` поверх `data/fsp_registry.json` (синтетические данные; файл перечитывается при изменении).

7.2 Привязка профиля

```
Кандидат вводит FSP ID ──► POST /api/candidate/fsp ──► registry.get(id)
                                                  │ e-mail записи == e-mail аккаунта?
                                              да ─┴─ нет → 403 `fsp_email_mismatch` (ID не найден →
404 `fsp_not_found`; уже привязан к другому аккаунту → 409 `fsp_taken`)
                                                  │
                                  сохранить fsp_id ► summarize(events) ► {top%, число мероприятий, си
ла 0..1}
```

Проверка совпадения e-mail нужна, чтобы нельзя было привязать чужой ФСП ID и «присвоить» чужие достижения.

7.3 Целевая схема (после MVP)

```
Браузер ──(Authorization Code + PKCE)──► Keycloak ФСП ──► id_token { sub, email, fsp_id }
Платформа (клиент OIDC) ── принимает токен, привязывает профиль по sub/fsp_id (ввод вручную
не нужен)
Платформа ── GET {FSP_API}/participants/{fsp_id}/events (service token) ──► кэш на 24 ч ──►
summarize()
```

Замена затрагивает два места: `verify_link` (проверка по токену вместо e-mail) и `FileRegistry` (HTTP-клиент с кэшем и таймаутами). Кандидат без истории ФСП (`events` пуст или записи нет) обрабатывается везде одинаково: сводка `None`, составляющая «ФСП» в оценке равна 0, в карточке — «истории ФСП нет», категория не меняется.

---

8. API

Интерактивная документация — **`/docs`** (Swagger UI), схема — **`/openapi.json`** (OpenAPI 3.0.3). Аутентификация — `Authorization: Bearer <JWT>` из `POST /api/auth/login`. Формат ошибок единый:

```
{"error": "grade_change_limited", "message": "Смена грейда доступна не чаще одного раза в 90 дней…", "details": {}}
```

8.1 Аутентификация и общие

Метод	Путь	Назначение
POST	`/api/auth/register`	Регистрация (роль, e-mail, пароль, `consent_pd`)
POST	`/api/auth/confirm`	Подтверждение e-mail по токену
POST	`/api/auth/resend`	Повторная отправка письма
POST	`/api/auth/login`	Вход, выдача JWT
GET	`/api/auth/me`	Текущий пользователь
DELETE	`/api/auth/account`	Удаление аккаунта и данных
GET	`/api/taxonomy`	Справочники: специализации, грейды, стек, вопросы опроса
GET	`/api/stats`	Публичная статистика платформы
GET	`/api/about`	Сведения о ФСП для подвала сайта (из `data/fsp_about.json`)
GET	`/api/health`	Проверка живости

8.2 Кандидат

Метод	Путь	Назначение
GET	`/api/candidate/me`	Сводное состояние: профиль, категория, ограничения смены грейда, шаги пути
PUT	`/api/candidate/profile`	Профиль и самоописанное резюме
POST	`/api/candidate/resume/parse`	Разбор загруженного PDF
GET	`/api/candidate/profile.pdf`	Профиль в PDF (с блоком «Подтверждено платформой»)
POST	`/api/candidate/survey`	Опрос
POST / GET	`/api/candidate/tests`	Начать попытку / история попыток
GET	`/api/candidate/tests/{id}`	Попытка (задания без ответов; после сдачи — разбор)
POST	`/api/candidate/tests/{id}/proctor`	Прокторинг: пакет событий и снимок камеры; в ответе риск, предупреждение, признак прекращения
POST	`/api/candidate/tests/{id}/finish`	Сдать ответы (и время по заданиям)
PUT	`/api/candidate/settings`	Публикация, приватность
POST / DELETE	`/api/candidate/fsp`	Привязать / отвязать ФСП ID

Метод	Путь	Назначение
GET	`/api/candidate/invites`, `/invites/{id}`	Приглашения (просмотр ставит статус «просмотрено»)
POST	`/api/candidate/invites/{id}/decision`	Принять / отклонить
GET	`/api/candidate/vacancies`, POST `/vacancies/{id}/apply`	Вакансии и отклик
GET	`/api/candidate/applications`	Мои отклики
GET / POST	`/api/candidate/microtasks`, `/microtasks/{id}/answer`	Микрозадачи
GET	`/api/candidate/export`	Выгрузка всех данных (152-ФЗ)
GET	`/api/candidate/vacancies/{id}`	Вакансия целиком: описание, требования, условия, компания
GET / PUT	`/api/candidate/resume`	Резюме внутри сервиса: опыт, проекты, образование, дипломы и сертификаты
POST	`/api/candidate/practice/next`, `/practice/{id}/answer`, `/practice/{id}/skip`	Тренировка: задание, ответ с разбором, пропуск
GET	`/api/candidate/practice/stats`, `/api/candidate/achievements`	Статистика тренировки; награды, очки и уровень

8.2a Общие для кандидата и работодателя

Метод	Путь	Назначение
PUT / DELETE	`/api/account/avatar`	Фото профиля / логотип компании (меню аккаунта)
GET	`/api/wallet`	Баланс, услуги своей роли, история операций
POST	`/api/wallet/topup`, `/api/wallet/buy`	Пополнение и покупка услуги
POST / GET	`/api/support`	Обращение в поддержку и ответы

8.3 Работодатель

Метод	Путь	Назначение
GET / PUT	`/api/employer/company`	Профиль компании
GET	`/api/employer/summary`	Показатели кабинета
POST / GET / DELETE	`/api/employer/needs`	Потребности
GET	`/api/employer/needs/{id}/matches`	Подбор по потребности: категории, кандидаты, баллы, «почему»
GET	`/api/employer/candidates`	Банк кандидатов с фильтрами и пагинацией

Метод	Путь	Назначение
GET	`/api/employer/candidates/{code}`	Карточка кандидата (псевдоним; контакты — только после принятия)
POST / GET	`/api/employer/invites`	Отправить приглашение (**вилка зарплаты обязательна**) / список со статусами
DELETE	`/api/employer/invites/{id}`	Отозвать приглашение
POST / GET	`/api/employer/vacancies`	Создать / список своих вакансий (город, занятость, требования, условия)
GET / PUT / DELETE	`/api/employer/vacancies/{id}`	Вакансия: просмотр, редактирование, удаление
PATCH	`/api/employer/vacancies/{id}/status`	Открыть / закрыть
GET	`/api/employer/vacancies/{id}/applications`	Отклики
POST	`/api/employer/applications/{id}/status`	Статус отклика
POST / GET	`/api/employer/microtasks`, `/microtasks/{id}/submissions`	Микрозадачи и ответы
POST	`/api/employer/assignments/{id}/rate`	Оценка ответа

8.4 Модератор

Дополнительно: `GET /api/moderator/support` и `POST /api/moderator/support/{id}/reply` — очередь обращений в поддержку.

Метод	Путь	Назначение
GET	`/api/moderator/queue?state=flagged|decided|all`	Очередь попыток на проверку
GET	`/api/moderator/attempts/{id}`	Журнал прокторинга попытки
GET	`/api/moderator/snapshots/{id}`	Снимок с камеры (JPEG)
POST	`/api/moderator/attempts/{id}/decision`	`clear` — снять отметку, `annul` — аннулировать

Всего 77 методов; параметры, схемы тел и примеры ответов — в Swagger UI.

---

9. Библиотеки и версии

Пакет	Версия	Назначение
Python	3.13 (3.11+ подойдёт)	Среда выполнения

Пакет	Версия	Назначение
Flask	3.1.3	HTTP-слой
Werkzeug	3.1.8 (зависимость Flask)	WSGI
pydantic	2.13.5	Валидация запросов, генерация JSON Schema
PyJWT	2.14.0	JWT
pypdf	5.9.0	Разбор PDF-резюме
reportlab	5.0.1	Генерация PDF
gunicorn	23.0.0	Боевой WSGI-сервер (Docker); не проверялся в среде разработки
SQLite, hashlib.scrypt, ipaddress, re, random	стандартная библиотека	БД, хеширование паролей, вычисление ответов заданий
fonts-dejavu-core	системный пакет	Кириллица в PDF

Фронтенд — без внешних библиотек и CDN. Версии закреплены в `requirements.txt`.

---

10. Запуск

10.1 Docker (рекомендуется)

```
git clone <репозиторий> fsp-talent && cd fsp-talent
export SECRET_KEY=$(python3 -c "import secrets;print(secrets.token_hex(32))")
docker compose up --build
# откройте http://localhost:8000      (Swagger UI: http://localhost:8000/docs)
```

При первом запуске (`SEED_DEMO=1` по умолчанию) база наполняется демо-данными: 70 кандидатов, 5 работодателей. Данные хранятся в томе `dbdata`.

10.2 Без Docker

```
python3 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
python3 scripts/seed.py          # необязательно: демо-данные
python3 run.py                   # http://localhost:8000
```

10.3 Демо-доступ

Пароль у всех демо-аккаунтов — `Demo12345`.

Роль	Логин	Что посмотреть
Кандидат	`demo.candidate@fsp-demo.ru`	Категория, результат теста, привязка ФСП
Работодатель	`demo.employer@fsp-demo.ru`	Банк кандидатов, «почему», приглашение

Роль	Логин	Что посмотреть
Модератор	`moderator@fsp-demo.ru`	Очередь отмеченных попыток, журнал, снимки (в демо — три отмеченных)
Свежий сценарий	зарегистрируйтесь в интерфейсе	В `DEV_MODE=1` ссылка подтверждения придёт в ответе и на экран

10.4 Конфигурация (переменные окружения)

Переменная	По умолчанию	Назначение
`SECRET_KEY`	случайный при старте	Ключ подписи JWT. **В продакшене задать обязательно**
`DB_PATH`	`data/app.db`	Файл SQLite
`DEV_MODE`	`1`	`1` — письма не уходят, ссылка в ответе; `0` — отправка по SMTP
`SMTP_HOST/PORT/USER/PASSWORD/FROM`	—	Почтовый сервер
`PUBLIC_URL`	`http://localhost:8000`	База для ссылок в письмах
`GRADE_COOLDOWN_DAYS`	`90`	Интервал смены грейда
`TEST_ATTEMPTS_PER_DAY`	`5`	Попыток в сутки
`TEST_TIME_LIMIT_MIN`	`90`	Лимит времени теста
`PASS_THRESHOLD`	не задан	Единый порог зачёта; без него действуют пороги по грейдам (0.65 / 0.65 / 0.70 / 0.80)
`PROCTOR_CAMERA_REQUIRED`	`1`	Без камеры тест не начать (`0` — отключить требование)
`PROCTOR_RETENTION_DAYS`	`30`	Срок хранения снимков с камеры
`FSP_ABOUT_FILE`	`data/fsp_about.json`	Сведения о ФСП для подвала сайта
`INVITES_PER_DAY` / `MICROTASKS_PER_WEEK`	`25` / `2`	Антиспам
`PAYMENTS_MODE`	`demo`	`demo` — пополнение зачисляется сразу, деньги не списываются; иначе пополнение отвечает 501, пока не подключён платёжный провайдер
`FSP_REGISTRY_FILE`	`data/fsp_registry.json`	Реестр ФСП

10.5 Тесты

```
python3 tests/runner.py          # 35 сквозных тестов (также совместимо с pytest)
python3 scripts/validate.py      # процедура валидации, §6
```

Тесты покрывают: регистрацию и подтверждение, разделение ролей, путь «опрос → тест → категория», уникальность тестов, запрет понижения грейда и ограничение смены, флаги подозрительного прохождения, привязку ФСП и пустую историю, объяснимость и скрытие контактов, видимость кандидатов, жизненный цикл приглашения и обязательную вилку зарплаты, изоляцию данных, вакансии и отклики, микрозадачи, PDF туда-обратно, выгрузку и удаление аккаунта, полноту OpenAPI, обязательность согласия и камеры для прокторинга, отметку, прекращение и аннулирование попытки модератором (категория снимается), каталог технологий и данные подвала.

---

11. Монетизация, награды, тренировка

Платные услуги (кошелёк в меню аккаунта; `app/services/shop.py`). Кандидату: «Поднять профиль» 299 ₽/7 дн., «Экспертный тест» 990 ₽ (одна попытка), значок Pro 199 ₽/30 дн. Работодателю: «Продвинуть вакансию» 490 ₽/7 дн., «Пакет приглашений +50 в сутки» 1490 ₽/30 дн., фильтр «только Expert» 1990 ₽/30 дн.

Принцип честности: платные услуги не меняют категорию — её присваивает только тест. Преимущество — небольшая и **подписанная** прибавка в подборке (`BOOST_BONUS` 0,04 к итоговому баллу, `EXPERT_BONUS` 0,05 к профильной части), она показана работодателю в разделе «почему в подборке». Экспертный тест — усложнённый (уровень выше Senior), проходит под тем же прокторингом.

Оплата. В режиме `PAYMENTS_MODE=demo` пополнение зачисляется мгновенно, реальные деньги не списываются. Для продакшена нужен платёжный провайдер (ЮKassa, CloudPayments и т. п.): эндпоинт пополнения заменяется на создание платежа и вебхук подтверждения; кошелёк, покупки и каталог уже готовы.

Награды (`app/services/rewards.py`): 18 значков за профиль, резюме (в т. ч. «Резюме 100%»), тесты, тренировку и карьеру; очки и 5 уровней. Видны только кандидату.

Тренировка (`app/services/practice.py`): задания из того же генератора, без таймера, камеры и категории; разбор сразу. На категорию и подбор не влияет, только на награды.

Резюме внутри сервиса: опыт, проекты, образование, дипломы и сертификаты; попадает в PDF. Работодатель видит его по настройкам приватности, а названия компаний, вузов и ссылки скрыты до принятия приглашения.

---

12. Развитие за пределами MVP

- **Защита от «фейковых» вакансий и рассылок.** Верификация компании (ИНН, домен e-mail), лимиты приглашений по репутации, жалобы кандидатов и «рейтинг ответа» (какую долю приглашений работодателя принимают), ручная модерация новых аккаунтов.
- **Интеграция с ATS** (Хантфлоу, Potok, Talantix и др.): передача принятых кандидатов в воронку, вебхуки статусов.

- **Усиление прокторинга:** видеозвонок с живым проктором для финалистов, распознавание второго устройства, потоковая запись по решению ФСП, калибровка весов и порогов по данным пилота.
- **Рост банка заданий:** новые специализации, уровень 6+ для senior, автоматический мониторинг утечки и «выбраковка» семейств.
- **PostgreSQL и очередь писем**, горизонтальное масштабирование (сейчас SQLite — один процесс).
- **Keycloak/OIDC ФСП** вместо проверки по e-mail (§7.3).

---

13. Известные ограничения

- Реестр ФСП — синтетический файл; интеграция проверена на контракте, не на реальном сервисе.
- Валидация — на модели; содержательную валидность тестов предстоит подтвердить на реальных данных (§6.5).
- Подсказка в реальном времени не устраняется: прокторинг в браузере обходится на уровне ОС (вторая камера, второй компьютер); веса и пороги риска экспертные, не калиброваны (§4.7).
- Контакты ФСП в подвале берутся из `data/fsp_about.json`; перед запуском в production их нужно сверить с актуальными данными федерации.
- Тест senior различает middle хуже остальных (§6.4).
- Пул концептуальных вопросов — самая уязвимая к утечке часть банка (§6.4).
- Оплата — демонстрационная: платёжный провайдер не подключён, платные бонусы в подборке выбраны экспертно и подлежат калибровке.
- SQLite и один процесс — достаточно для демонстрации и пилота, не для высокой нагрузки.
- Парсер PDF — эвристический: надёжно извлекает e-mail, телефон, стек из словаря; ФИО, опыт и роли — по шаблонным подписям, результат всегда проверяется кандидатом перед сохранением.
- Образ Docker не собирался в среде разработки (закрытый доступ к PyPI); версии зависимостей закреплены по установленным и проверенным локально, `gunicorn` — по номеру релиза.

14. Изменения сборки 2026.10.11-2

14.1 Кандидат

- **Отклик**: `PATCH /api/candidate/applications/{id}` — изменить сопроводительное сообщение; `DELETE` — отозвать отклик. После отзыва можно откликнуться повторно; отозванный отклик не раскрывает профиль работодателю.
- **Конструктор резюме** (`GET/PUT /api/cabinet/resume`): опыт работы, образование (с уровнем), языки, ссылки, личные данные. Ответ содержит `advice` — подсказки по заполнению (заполненность, измеримые результаты, длина описаний). Выгрузка в PDF: `GET /api/candidate/profile.pdf` , параметр `personal=1` добавляет личные данные.

- **Личные данные** (возраст, гражданство, семейное положение, дети, воинский учёт, водительские права) по умолчанию скрыты (`privacy.show_personal = false`). Работодатель получает их только по явному согласию кандидата; в интерфейсе приведена ссылка на ст. 64 ТК РФ о недопустимости необоснованных отказов по таким признакам.
- Меню профиля работает: профиль и резюме, награды, задания, кошелёк, настройки, поддержка.

14.2 Работодатель

- В результатах кандидата кнопка «Подробнее» (вместо «Почему»), в подборе — «Поиск» (вместо «Уточнить»).
- Просмотр резюме кандидата и выгрузка `GET /api/employer/candidates/{code}/resume.pdf` (контакты замаскированы до раскрытия профиля).
- Расширенная карточка кандидата: опыт, образование, языки, ссылки и разрешённые личные данные.
- Приглашения: зарплатная вилка вводится любыми числами. Статус (`sent → viewed → accepted / declined / withdrawn`) ведёт система, а **стадию интереса** — работодатель: `PATCH /api/employer/invites/{id}/stage` (на рассмотрении, собеседование, тестовое, оффер, принят и др.). История стадий хранится в `stage_log` , кандидат видит текущую стадию.
- Вакансии: редактирование и удаление через собственное окно подтверждения, выбор технологий из каталога (215 технологий), поднятие вакансии.

14.3 Каталог и главная

- 12 специализаций (веб, мобильная разработка, ИБ, Embedded, игры и графика, аналитика и др.); составные специализации описаны в `taxonomy.SPEC_BASES` , тесты для них собираются из семейств заданий баз.
- Публичная витрина `GET /api/public/showcase` (направления, кандидаты, вакансии) для главной страницы. Контакты ФСП и ссылки на направления — в `data/fsp_about.json` .

14.4 Прокторинг

Добавлены сигналы `phone_detected` и `multi_person` с весами и потолками в `services/proctor.py` . Детектор COCO-SSD (TensorFlow.js) загружается в браузере из CDN; без интернета он отключается без ошибок, остальные сигналы (смена вкладки, полный экран, снимки камеры) работают. Автоматический сигнал — лишь основание для проверки модератором, а не окончательное решение.

14.5 Эксплуатация

- Версия сборки `BUILD = 2026.10.11-6` отдаётся в `/api/public/health` и сверяется фронтендом: при несовпадении показывается предупреждение о старом сервере. Для html/css/js отключено кэширование.
- Оформление: логотип ФСП, шрифты TDA Text и JetBrains Mono (лицензии OFL лежат рядом с файлами в `app/static/fonts`).
- Платежи остаются демонстрационными (`PAYMENTS_MODE=demo`).

- Всего HTTP-эндпоинтов API: 86; сквозных тестов: 35.

14.6 Сборка 2026.10.11-3

- **Логотип** в шапке больше не обрезается (область `viewBox` расширена, показываются все три элемента знака).
- **Потребности**: `PUT /api/employer/needs/{id}` и кнопка «Изменить» в разделе «Мои потребности»; рядом на обзоре выводится список вакансий с кнопкой «Редактировать».
- **Отклики на вакансии**: добавлен статус `call_passed` («успешно прошёл созвон», зелёный) — кандидат считается принятым. Порядок статусов: просмотрено → приглашение на разговор → успешно прошёл созвон → отказ.
- **Окно приглашения**: сообщения об ошибках теперь показываются внутри открытого окна (раньше они рисовались под ним и были не видны). В банке кандидатов кнопка «Пригласить» доступна всегда, а не только внутри раскрытого «Подробнее». Если в вакансии или потребности нет описания, текст предложения собирается автоматически (раньше пустое поле блокировало отправку).
- **Источник приглашения**: в списке выбираются и вакансии, и потребности; по умолчанию подставляется самая свежая опубликованная вакансия. В ответах `GET /api/employer/invites` добавлены `vacancy_title` и `need_title`, в списке приглашений показывается, по какой вакансии или потребности оно отправлено.
- **Демо-данные**: заранее созданных приглашений в базе нет — только те, что отправлены вручную.
- **Возврат приглашения**: `POST /api/employer/invites/{id}/restore` возвращает отозванное приглашение в статус «отправлено» (если нет другого активного приглашения этому кандидату); в разделе «Приглашения» — кнопка «Вернуть приглашение».
- **Задания по FSP ID**: в `POST /api/employer/microtasks` новое поле `fsp_ids` (список или строка через запятую/пробел). Задание сразу назначается кандидатам с привязанным FSP ID; ответ содержит `delivered` и `not_found`. Адресное задание не раздаётся другим кандидатам; у адресата оно помечено «Лично для вас».

14.7 Сборка 2026.10.11-4

- У отозванного приглашения не показывается метка «Отправлено» — только «Отозвано».
- Отклики на вакансию: работодатель выбирает только «Принят» (`hired`, зелёный) или «Отказ»; прежние статусы (`viewed`, `invited_to_talk`, `call_passed`) остаются допустимыми в API для совместимости, но в интерфейсе не показываются.
- Когда в «Приглашениях» этап кандидата меняется на «Принят на работу» и приглашение привязано к вакансии, кандидат автоматически попадает в отклики этой вакансии со статусом «Принят» и видит его у себя.
- `DELETE /api/employer/invites/{id}/permanent` — удалить кандидата из приглашений навсегда (без возврата); в интерфейсе — кнопка «Удалить кандидата» с подтверждением.
- Из списка потребностей на обзоре убрана кнопка «Изменить» (эндпоинт `PUT /api/employer/needs/{id}` сохранён).

14.8 Сборка 2026.10.11-6

- **«Принят на работу» → отклики вакансии.** При смене этапа на «Принят на работу» кандидат попадает в «Вакансии → Отклики» со статусом «Принят» (там остаются выбор «Принят» / «Отказ»). Если приглашение не привязано к вакансии, интерфейс просит выбрать вакансию (`vacancy_id` в `POST /api/employer/invites/{id}/stage`); без вакансии сервер отвечает 422. Ответ содержит `placed_in_vacancy` .
- **Оформление.** Фиолетовая цветовая схема интерфейса сохранена (экспериментальная перекраска в синий и красный цвета логотипа отменена по просьбе заказчика). Логотип ФСП, шрифты TDA Text и JetBrains Mono остаются; для символов, которых нет в TDA Text (например, «·»), подставляется JetBrains Mono. Цвета заданы переменными в начале `app/static/app.css` .

14.9 Сборка 2026.10.11-6

Возвращено прежнее фиолетовое оформление интерфейса; весь функционал сборки 2026.10.11-5 сохранён без изменений.

14.10 Сборка 2026.10.11-7

В «Вакансии → Отклики» у каждого кандидата, рядом с «Принят» и «Отказ», появилась кнопка «Удалить» (с подтверждением): `DELETE /api/employer/applications/{id}` . Отклик исчезает из списка вакансии насовсем и у кандидата; откликнуться заново он может. Чужие отклики удалить нельзя (404).